

The Fintech Split

Why 17,022 customers live on the fintech book while three and a half million of their ledger rows never left the field book — and what it takes to finish the move, lock the door behind it, and let a service keep it that way.

Verified live

Every figure below was read from `Serviceconnect` at `100.72.35.56,4230` on 8 September 2026 through the read-only SQL relay. Nothing has been written.

§ A

What the first migration actually did

And the precise shape of what it left behind

The move happened in a single batch on **2 August 2026 at 19:43:07**. It is recorded in full: `BorrowerEntityMigrationBackup_20260802` holds 17,016 rows, every one of them `3002` → `3005`. That backup table is the reason this is recoverable rather than archaeology.

It moved two things correctly and stopped:

TABLE	CARRIES <code>ENTITYID</code>	MOVED ON 2 AUG	ROWS WRONG TODAY
<code>Borrowers</code>	yes	yes	0
<code>Loans</code>	yes	yes	1
<code>Journals</code>	yes	no	1,402,813
<code>LoanGraduationHistory</code>	yes	no	1,211,692
<code>CustomerStatement</code>	yes	no	603,408
<code>loanSchedule</code>	yes	no	298,228
<code>ManagedLoans</code>	yes	no	1,144
<code>loanChargesSchedule</code>	yes	no	1,075
<code>STKPaymentRequests</code>	yes	no	22

The three and a half million rows, to scale

Each bar is rows whose own `EntityId` still says `3002` while the loan or the customer they belong to says `3005`.

Journals		1,402,813
LoanGraduationHistory		1,211,692
CustomerStatement		603,408
loanSchedule		298,228
ManagedLoans		1,144
loanChargesSchedule		1,075
STKPaymentRequests		22
Total mis-stamped		3,518,382

`Journals` is the only one of these with both a `BorrowerId` and a `LoanId`, and it is driven off the customer: 289,267 of its rows carry no fintech loan at all — registration fees and other non-loan postings that still belong to the person. Exactly one row disagrees with itself (a 3005 customer, a 3002 loan). The script excludes and reports it rather than picking a side.

THIS IS THE REPORT LEAK

You asked why Micromart Africa's reports still show fintech customers. This is the whole answer, and it is not a reporting bug. Any report that reads `Journals`, `CustomerStatement` or `loanSchedule` and filters on *that table's* `EntityId` is being told, correctly, that 3.5 million fintech ledger rows belong to the field book. Restamp the rows and the reports separate on their own — no report code has to change.

Where the customers are standing

All 17,022 fintech borrowers are still attached to the relationship officers who onboarded them. Entity 3005 has exactly one branch — `UnitId 129`, "Main Office" — and most are already in it, but the officer link never moved.

POSTURE INSIDE <code>3005</code>	BORROWERS	SHOULD BE
Total on the fintech book	17,022	—
In <code>EntityUnit 129</code> (Main Office)	16,264	all 17,022
In a field branch that belongs to 3002	758	0
Distinct <code>EntityAgent</code> values (ROs)	167	2
Under Geoffrey Njane (<code>9098</code>)	4	the migrated book
Under Emmanuel Birgen (<code>9096</code>)	1	console / shelf origin

Two blockers on the destination itself. Geoffrey's user record 9098 has `UserStatus = 0` — the account is **disabled**. And both his record and yours carry `OrganizationUnit = 1`, which is 3002's "Head Office", not 3005's "Main Office" (129). Parking 17,022 customers under a disabled user in the wrong branch would replace one mess with a quieter one, so both need fixing before any re-parenting runs.

The re-onboarding, in numbers

Exactly what you described is measurable. Since the migration date, ROs have created 76 fresh borrower records on 3002 for people whose identity already sat on 3005 .

DUPLICATION ACROSS THE TWO BOOKS	COUNT
National IDs present on both books	200
Phone numbers present on both books	52
Records created on 3002 after 2 Aug matching a 3005 identity	76
Duplicate borrower records sitting on 3002	201
— of those, with no open loan → merge by script	143
— of those, with an open loan → needs a person	58
Identities carrying an open loan on both books	1

58 of them have live money. This is the one number that changed the plan. Merging a duplicate means moving its loans to the surviving record — and for 58 customers that would move an open balance out of Micromart Africa's portfolio and into Fintech's, and change whose collections queue they sit in. That is an accounting event, not a data fix, so the script refuses to do it and prints the list instead. The usual answer is to leave them: let the loan run to term on 3002, and the dormancy service moves the customer 60 days later. That is precisely the path it exists to provide.

A worked example of the pattern:

NationalID 11198183 – one person, two records

- ID 18797 3005 unit 129 created 2022-10-14 2 loans, all cleared
- ID 168525 3002 unit 130 created 2026-08-13 1 loan, KES 2,974 open

The second record was created eleven days AFTER the migration, by an RO, with a different phone number. A phone-only uniqueness rule would not have caught it. The National ID rule does – which is why the lock needs both.

§ B

Your eight questions, answered

Numbered as you asked them

4 · Why the PIN reset SMS said "Dear Mr" and came from entity 3002

Both halves are one stored procedure. `sp_resetBorrowerPin` selects the borrower's real entity into `@entityId` — and then never uses it:

```
SELECT @FirstName=firstName, @Phone=PhoneNumber, @entityId=EntityId
FROM Borrowers WHERE ID=@BorrowerId

SET @sms='Dear '+@FirstName+' Your '+@EntityName+' Pin has been reset. ...'

INSERT INTO Notifications.dbo.SMS(smsMessage, smsto, EntityId, ...)
VALUES(@sms, @Phone, 3002, GETDATE(), 0, getdate(), 5)
      ^^^^ hard-coded literal - @entityId is discarded
```

So every borrower PIN reset in the database is stamped `3002`, fintech customers included. Your own reset on 29 August is in the outbox as `ID 2767547`, body "Dear Emmanuel Your **MICROMART FINTECH** Pin has been reset" — correct lender name, wrong entity stamp. That single literal is the fix.

The "Dear Mr" is not a templating failure — the concatenation works. The row you pasted is from **July 2024**, and the borrower record behind it had `firstName` literally set to `'Mr '` (note the double space in "Dear Mr Your", which is `'Dear ' + 'Mr ' + ' Your '`). An RO had typed the title into the first-name field. Your current record, ID `170497`, reads "Emmanuel" and renders correctly.

Worth knowing: outbound SMS does not live in `Serviceconnect.dbo.SMS` — that table stopped in May 2023. The live outbox is `Notifications.dbo.SMS`, a different catalogue on the same server. Anything that greps the wrong table will conclude, wrongly, that no SMS is being sent.

5 · The 400 from `/api/portal/otp`

Reproduced from here. The endpoint has exactly one 400 you can trip from that screen, and it is not the phone number:

REQUEST	RESPONSE	STATUS
<code>{phone}</code> , no <code>lenderSlug</code>	"Choose a lender."	400
<code>lenderSlug: "micromart-fintech"</code>	"Choose a lender."	400
<code>lenderSlug: "micromart"</code>	<code>delivered: false</code>	200
<code>phone: "0758517032"</code> + <code>slug</code>	<code>delivered: false</code>	200

`micromart` is the only slug that resolves; `micromart-fintech` is a ServiceSuite registry name, not an `Org.slug`, and the welcome screen is sending something that misses. That is the 400.

But fixing the slug only buys a 200. The second, larger problem is `delivered: false` — `hasSmsProvider(orgId)` is false in production, so no code is ever dispatched. Your instruction to send it the way the other PWA does is the right fix and it has a concrete shape: stop asking a generic SMS provider and write into `Notifications.dbo.SMS` with `EntityId 3005` and `SmsProviderId 5`, exactly as `sp_restBorrowerPin` does. That is what puts Micromart's sender ID on the message — the sender ID is a property of the entity's Africa's Talking credentials, not of the calling app.

6 · Why your reset came from Techcrast, at entity 7, in the shared database

THE DEPLOYED APP IS NOT THIS CODEBASE

I downloaded the live bundle from `pwa.servicesuitecloud.com` and read it. It contains 42 API calls, and every one of them points at `https://live.testapps.co.ke/MicromartAPI/...` — the shared platform on `213.148.17.198`. Not `micromartafrica.co.ke`. It also carries `EntityId: 7` as a literal.

So `pwa.servicesuitecloud.com` is serving a build made *before* the `src/lib/entity.js` fix in `C:\GIT\micromart-client-pwa`. Your local repo is already correct and already points at Micromart. It has simply never been deployed to that origin.

That explains the message end to end: "SERVICE SUITE TEST AREA", entity 7, Techcrast's sender ID, and a row that only exists in the `.198` database — you reset a password on a different company's book.

Which products those are

The three cards in your screenshot — Salary Advance, Weekly Business Loan, 4 weeks Product — are **entity 7's products on the shared test platform**. They do not exist anywhere in Micromart's database. Micromart Fintech's real shelf is three products, and this is all of it:

ID	PRODUCT	MIN	MAX	TERM	BRANCHES
30219	Micro Eazy	10,901	100,000	10 weeks	0,129
30220	Micro Eazy Monthly	10,901	100,000	2 months	0,129
30221	Micro Chap Chap	5,000	10,900	10 weeks	129

Note the `EntityUnits` column: a borrower must be in branch `129` to be offered any of them. The 758 fintech borrowers sitting in a field branch are shown an empty shelf — and the same re-parenting that fixes the RO tie fixes that too.

Your proposed lookup is the right one and matches what the data supports: query `Borrowers` by `AccountNo` or `NationalID`; two books → refuse and tell them to contact admin; one book → scope the session to it; neither → offer registration. There are 52 phone numbers and 200 IDs that would hit the "both books" branch today, which is why § C phase 3 merges them before this ships.

8 · The USSD error, and the PIN

Taking the second part first, plainly: **the PIN cannot be decrypted**. `Borrowers.UssdPin` holds a BCrypt hash — yours is a 60-character `$2b$` — and BCrypt is a one-way function. There is no key and nothing to reverse. What I can tell you is that `6836` is from that July 2024 SMS and is long dead: your PIN was reset on 29 August 2026, and the outbox row for it (`ID 2767547`) carries the current value. A fresh reset from the console is the clean way in.

The technical-error path is a real defect, and the message you saw is Africa's Talking's own text — it is what the gateway shows a subscriber when the callback returns a non-200. `ServiceController.cs` has **no global exception handler**, so any throw becomes a 500 and the customer is told the service is broken. The most likely throw is in `ValidatePin` :

STORED <code>USSDPIN</code> SHAPE	BORROWERS	BCRYPT VERDICT
<code>\$2b\$</code> , 60 chars	12,941	<code>valid</code>
<code>\$2a\$</code> , 60 chars	2,528	<code>valid</code>
<code>\$2a\$</code> , 50 chars	40	<code>truncated → throws</code>

Those 40 hashes are truncated. `BCrypt.Verify` raises `SaltParseException` on them, nothing catches it, and the customer gets the gateway's error instead of "Invalid PIN". Two fixes, both small: wrap the handler so it can never return a non-200, and null the 40 broken hashes so those customers fall into the existing set-your-PIN flow.

One thing I could not confirm and should not guess at: the analysis report of 2 September describes the channel as `*384*NNNN#` , and you are dialling `*483*490#` . Those are different service codes. Before I change `ATusersUssdApI` I need to know that `*483*490#` actually routes to that deployment — otherwise I would be fixing a service nobody is dialling. It also matters for the wider defects: the same report already documents that the PIN is read with no entity predicate, so setting a PIN on one book overwrites it on every book sharing that number.

7 · A demo account for the Micro Eazy app

I can create one, but it will not let you sign in yet, and I would rather say so than hand you credentials that fail. `C:\GIT\MICRO_EAZY\micro-eazy-app` has a `SignIn.tsx` screen, but its own `DEPLOY.md` records that the OTP calls are not wired to it — and the endpoint it would call is the one from question 5 that currently returns `delivered: false` . So the account and the sign-in path have to land together. Both are in phase 5.

§ C

The plan

Seven phases, each one gated on the last

What this machine cannot do. The relay reports `"writes": false` — it will read Micromart's database all day and refuse every write. That is its correct default and I am not proposing to change it. Every script below is delivered to you as reviewed SQL that *you* run, inside an explicit transaction, against a backup table. A second constraint: `ReportsController.cs` is ACL-locked to another account on this box and cannot be read or edited here — which is survivable, because § A shows the report fix is in the data, not in that file.

0

Snapshot, before anything

A full backup of `Serviceconnect`, plus a `FintechRemediationBackup_20260908` table capturing every row key and old `EntityId` the scripts will touch. Nothing in phases 1–4 is irreversible while that table exists — that is what makes the script reversible in the sense you asked for.

1

Fix the destination

Enable Geoffrey's account (`UserStatus 1`) and move both `9098` and `9096` to `OrganizationUnit 129`. Small, safe, and everything downstream parks customers on it.

2

Restamp the 3.5 million orphaned rows

Set `EntityId = 3005` across all seven child tables — batched in chunks of 5,000 with a wait between, so a live system never sees a long lock. This is the phase that separates the reports.

Three of these tables are **heaps with no index at all**, so each gets a temporary index on its identity column, built `WITH (ONLINE = ON)` — this server is Enterprise — and dropped afterwards. Without it, every batch would rescan a 6.8-million-row heap.

3

Re-parent the customers, then merge the duplicates

All 17,022 fintech borrowers to `EntityUnit 129` and `EntityAgent 9098`, except those whose origin is the console or the shelf, which go to `9096`. Then fold the 200 duplicate identities together under the policy you choose below.

Note: 143 merge by script; the 58 with open loans are printed for Geoffrey.

4 The hard lock on Borrowers

A filtered unique index on normalised `NationalID` and another on `PhoneNumber`, spanning *both* entities, plus an `INSTEAD OF INSERT` trigger that returns a readable error naming the entity the customer already sits on. Every channel — ServiceSuite, the RO app, USSD, the LMS console, Micro Eazy — hits the table, so the table is the only place a rule cannot be bypassed. This is why it belongs in the database and not in five applications.

Gated: phase 3 must clear first — the index cannot be created while duplicates exist.

5 Repair the channels

The `3002` literal in `sp_restBorrowerPin`. The OTP slug and its delivery through Micromart's sender ID. A deploy of the corrected `micromart-client-pwa` onto `pwa.servicesuitecloud.com`. The USSD exception handler, the 40 truncated hashes, and the entity predicate on the PIN. Your demo account, once sign-in answers.

6 The dormancy service

A SQL Agent job at 02:30 daily. Finds borrowers on `3002` whose loans are all cleared and whose last loan matured more than 60 days ago, moves the customer *and every child row* to `3005` under Geoffrey in Main Office, and sends the SMS and email. **568 customers** qualify today — that is the backlog it clears on its first run.

Safe by default: the procedure dry-runs unless called with `@DryRun = 0`.

The welcome message

Sent by phase 6 on every move, through Micromart's own sender ID so it arrives under the name the customer already trusts. Draft:

Dear {FirstName}, your Micromart account has moved to Micromart Fintech.
You can now borrow from home — no branch visit, no agent.

App portal.servicesuitecloud.com
USSD *483*490#
PIN {pin}

Apply for your next loan any time. PIN yako ni siri yako.

WHY THE MESSAGE CARRIES NO PIN

`UssdPin` is a BCrypt hash, and BCrypt cannot be computed in T-SQL. A SQL Agent job therefore cannot mint a working credential, and writing a plain number into that column would reproduce the exact defect that breaks the USSD service today — `BCrypt.Verify` throws on a non-hash, which becomes a 500, which the customer reads as "technical problems".

So the service sets the PIN to `NULL` and the message sends them to `*483*490#`, where the USSD service's own set-your-PIN screen already handles it. No PIN travels by SMS, which is the better answer anyway. If you want a pre-set PIN in the message, the service has to be the .NET worker rather than a SQL Agent job — that is the one thing this option cannot do.

Email reaches almost everyone here, which was worth checking rather than assuming: 566 of the 568 customers in today's backlog have an address on file. It goes into the existing `Mails` outbox, so it leaves by the same route as every other Micromart email.

§ D What is on disk now

Eleven files, written against the answers you gave

All of it is in `C:\GIT\MICRO_EAZY\fintech-migration\`, to run in SSMS against `Serviceconnect`. Nothing has been executed.

FILE	DOES	ROWS
00-preflight.sql	Read-only assertions. Run first, keep the output.	—
01-fix-destination.sql	Enables Geoffrey, moves both users to Main Office	2
02-restamp-children.sql	The ledger rows, batched, with online helper indexes	3,518,382
03-reparent-borrowers.sql	Every fintech borrower to unit 129, off their RO	17,022
04-merge-duplicates.sql	Merges 143; prints the 58 that need a person	143
05-hard-lock.sql	The identity lock, with a self-test that must fail	—
06-fix-sp-restBorrowerPin.sql	Removes the hard-coded 3002	—
07-repair-broken-pins.sql	Clears the truncated hashes	40
10-dormancy-service.sql	Log table, procedure, daily 02:30 job	—
ROLLBACK.sql	Undoes 01–07, newest first	—
README.md	The runbook, the gates, what is still app-side	—

The gates the scripts enforce on themselves

These are not documentation. Each one is a check in the script that stops it running, because the order matters and a runbook someone reads at 2am is not a safety mechanism.

- 03 refuses while Geoffrey is disabled or still in unit 1 — otherwise 17,022 customers are parked on a dead account.
- 05 refuses while active duplicate identities remain, and prints the ones in the way. The 58 are among them: the lock is a promise the data has to be able to keep.
- 10 re-checks the destination on every run, not just at install, so the job cannot quietly start filing customers under a disabled user six months from now.
- 10 dry-runs by default. @DryRun is 1 unless the job overrides it.

Still outstanding — application work, not database work

None of the scripts touch these. They are the other half of getting you into the app, the USSD and the console on the fintech book.

ISSUE	WHERE IT LIVES
OTP slug mismatch, and hasSmsProvider false so nothing is ever delivered	connected-suite · api/portal/otp
Deploy the corrected PWA over the live.testapps.co.ke build	micromart-client-pwa
Global exception handler; entity predicate on the PIN read and write	ATusersUssdApI
Wire sign-in, then the demo account becomes usable	micro-eazy-app

One thing to confirm before anyone edits the USSD service. The 2 September channel analysis describes the dial string as *384*NNNN#. You are dialling *483*490#. Those are different service codes, and I could not verify from here which deployment answers yours. Fixing the wrong one would look like fixing nothing.